

Name: Phoebe Zhang  
Section: 03

### Question 1: Heuristic design and admissibility

Steps:

1. Count every letter position that is different from the initial word to the goal word.
2. Changing the letter that is different at least once to get the goal word is an unavoidable remaining cost.
3. It cannot overestimate the true optimal remaining cost because every letter that's different must be changed at least once to get to the goal word. A letter can be changed more than once to get to the goal word.

Scrabble:

1. For every letter position that's different from the goal, add up the scrabble cost of the letter that is being swapped in.
2. The letter that is placed into a position has to cost the scrabble cost of that letter is an unavoidable remaining cost.
3. It cannot overestimate the true optimal remaining cost because using the sum of the scrabble cost for the different letters position is the minimum.

Frequency: (Similar to steps)

1. Count every letter position that is different from the initial word to the goal word.
2. Changing the letter that is different at least once to get the goal word is an unavoidable remaining cost.
3. It cannot overestimate the true optimal remaining cost because every letter that's different must be changed at least once to get to the goal word. A letter can be changed more than once to get to the goal word. Also the frequency can't be negative.

### Question 2: Does the heuristic actually help?

| Word pair  | Cost measure | UCS cost | UCS nodes expanded | A* cost | A* nodes expanded | Reduction |
|------------|--------------|----------|--------------------|---------|-------------------|-----------|
| WHY -> ASK | Step         | 5.00     | 773                | 5.00    | 16                | 97.9%     |
| WHY -> ASK | Scrabble     | 9.00     | 765                | 9.00    | 13                | 98.3%     |
| WHY -> ASK | Frequency    | 38.86    | 808                | 38.86   | 755               | 6.6%      |

|              |           |       |      |       |     |       |
|--------------|-----------|-------|------|-------|-----|-------|
| BEAR -> DUCK | Step      | 4.00  | 691  | 4.00  | 6   | 99.1% |
| BEAR -> DUCK | Scrabble  | 11.00 | 2981 | 11.00 | 6   | 99.8% |
| BEAR -> DUCK | Frequency | 38.24 | 1084 | 38.24 | 720 | 33.6% |

1. UCS and A\* always return the same optimal cost.
2. For the WHY -> ASK word pair, the scrabble A\* produce the largest percentage reduction in expanded nodes (98.3%). For the BEAR -> DUCK word pair, it was scrabble (99.8%).
3. For the WHY -> ASK word pair, the frequency was least informative because it had the smallest reduction percentage (6.6%). For the BEAR -> DUCK word pair, it was frequency (33.6%).
4. For steps, the heuristic uses the number of wrong letters which basically tells A\* the true cost since it is using the same counting method. This is the same for scrabble but instead of counting the wrong letters it sums up the scrabble cost of the goal letters. On the other hand, frequency uses the same method with steps but the equation is  $1 + \text{rarity}(w)$  but the rarity isn't accounted for.

### Question 3: Reversibility and direction-dependent costs

Steps:

1. Yes
2. Actions are reversible and since each cost is just adding 1 for every letter position that is different, it makes the optimal cost equal.

Scrabble:

1. No.
2. When the actions are reversed, the order of the path is different making the sum different.
3. DUCK BEAR scrabble has a cost of 6 while BEAR DUCK has a cost of 11.

```
phoebezhang@Mac code_skeleton % python3 dcsolve.py DUCK BEAR scrabble
dc(duck, bear, scrabble) cost:6.00; time:0.000; solution:duck buck beck beak bear; deltas:[0.0, 0.0, 0.0, 0.0]
phoebezhang@Mac code_skeleton % python3 dcsolve.py BEAR DUCK scrabble
dc(bear, duck, scrabble) cost:11.00; time:0.001; solution:bear beak beck buck duck; deltas:[0.0, 0.0, 0.0, 0.0]
```

Frequency:

1. No
2. When the actions are reversed, it changes the order of the path making the rarity being summed different.
3. dog cat frequency has a cost of 23.02 while cat dog has a cost of 21.96.

```
phoebezhang@Mac code_skeleton % python3 dcsolve.py dog cat frequency
dc(dog, cat, frequency) cost:23.02; time:0.131; solution:dog don dan can cat; deltas:[-20.022, -13.737, -8.354, -6.887]
phoebezhang@Mac code_skeleton % python3 dcsolve.py cat dog frequency
dc(cat, dog, frequency) cost:21.96; time:0.343; solution:cat can dan don dog; deltas:[-18.955, -16.489, -11.106, -5.82]
```

## Question 4: LLM heuristic audit

### 1. Model: Claude

#### Setup and notation

Let  $n$  be the current state (a word of length  $L \in \{3, 4\}$ ) and let  $g$  be the goal word (also length  $L$ ). Write

- $h_0(n) = \#\{i : n_i \neq g_i\}$  — the Hamming distance to the goal (baseline heuristic),
- $\text{rarity}(w) \geq 0$  — the rarity value stored in the dictionary for word  $w$ ,
- $\text{cost}(w) = 1 + \text{rarity}(w)$  — the cost of the action that *enters*  $w$ .

Two facts about the search space matter here:

1. **Every action changes exactly one letter**, so no single action can repair more than one mismatched position. Hence any path from  $n$  to  $g$  (optimal or not) uses **at least  $h_0(n)$  actions**.
2. **The last action on any path that reaches  $g$  is necessarily "enter  $g$ ,"** and its cost is exactly  $\text{cost}(g) = 1 + \text{rarity}(g)$  — a known constant, not an estimate, since  $g$  is fixed and its rarity is in the dictionary.

#### Proposed heuristic

Let

$$m_L = \min_{w \in \text{Dict}, |w|=L} \text{rarity}(w)$$

be the smallest rarity value among all length- $L$  dictionary words (precomputed once per problem instance —  $O(|\text{Dict}|)$ ).

Define, for  $n \neq g$ :

$$h_1(n) = (h_0(n) - 1)(1 + m_L) + (1 + \text{rarity}(g)),$$

and  $h_1(g) = 0$ .

Intuitively:  $h_0(n) - 1$  of the remaining actions are "ordinary" letter-fixes, each costing at least  $1 + m_L$  (the cheapest possible entry cost for any legal word of this length); the *final* action is forced to be "enter  $g$ ," whose cost we know exactly,  $1 + \text{rarity}(g)$ .

## Admissibility

Take any path from  $n$  to  $g$  with actual actions  $w_1, w_2, \dots, w_k$  (so  $w_k = g$ ). By fact (1),  $k \geq h_0(n)$ . Its true cost is

$$\sum_{j=1}^k (1 + \text{rarity}(w_j)) = \underbrace{\sum_{j=1}^{k-1} (1 + \text{rarity}(w_j))}_{\geq (k-1)(1+m_L)} + (1 + \text{rarity}(g)) \geq (h_0(n) - 1)(1 + m_L) +$$

using  $\text{rarity}(w_j) \geq m_L$  for every intermediate word and  $k - 1 \geq h_0(n) - 1$ . Since this holds for every path to the goal, it holds for the optimal path, so  $h_1(n) \leq h^*(n)$ :  **$h_1$  is admissible.** (For  $n = g$ ,  $h_1(g) = 0 = h^*(g)$  trivially.)

## Why it is strictly more informative

Since  $m_L \geq 0$  and  $\text{rarity}(g) \geq 0$ ,

$$h_1(n) = h_0(n) + (h_0(n) - 1)m_L + \text{rarity}(g) \geq h_0(n) \quad \text{for all } n,$$

with equality only in the degenerate case  $m_L = \text{rarity}(g) = 0$ . In any dictionary where at least one length- $L$  word (in particular the goal itself) has positive rarity,  $h_1(n) > h_0(n)$  for **every** non-goal state  $n$  — strictly more informative than  $h_0$  everywhere it matters, while never exceeding the true remaining cost.

## Why not just add $\text{rarity}(n)$ ?

A tempting but *incorrect* variant would fold in the rarity of the current word itself. That's wrong here:  $n$  has already been entered by a *previous* action — its cost is sunk, not part of the remaining path cost. Including it would double count and risks inadmissibility if reused carelessly (e.g., along a re-entrant path).  $h_1$  only ever charges for actions that still need to happen.

## Remarks

- $h_1$  costs essentially nothing extra to evaluate at each node beyond  $h_0$ :  $m_L$  is computed once, and  $\text{rarity}(g)$  is a single dictionary lookup.
- $h_1$  can be tightened further if more structure is known (e.g., if you can additionally lower-bound the rarity of words matching the goal in at least one already-correct position), but  $h_1$  already captures the two pieces of rarity information that are "free" to exploit without guessing which intermediate words a path will use: a global floor  $m_L$  on every step, and the exact, unavoidable cost of the terminal step.
- $h_1$  is also consistent (triangle inequality holds edge-by-edge) by essentially the same argument used for admissibility, so it can be used safely with A\* without needing re-expansion.

2. For  $n \neq g$ :  $h_1(n) = (h_0(n) - 1)(1 - m_L) + (1 + \text{rarity}(g))$  and  $h_1(g) = 0$

```

def h(self, node):
    state = node.state

    different = []

    for i in range(len(state)):
        if state[i] != self.goal[i]:
            different.append(i)

    if self.cost == 'steps':
        return len(different)

    if self.cost == 'scrabble':
        total = 0

        for i in different:
            goal_letter = self.goal[i]
            total = total + scrabble_cost[goal_letter]

        return total

    if self.cost == 'frequency':
        if state == self.goal:
            return 0

        rarity_goal = dictionary[self.goal]

        return (len(different) - 1) * (1 + self.m_L) + (1 + rarity_goal)

```

3. For every action to swap the letters out to match the goal word, it would be  $h_0(n)$ .  
For the last action of reaching the goal word, it would cost  $1 + \text{rarity}(\text{goal})$ . The other moves would cost at least  $1 + m_L$  since it would produce some legal words.
4. The proposal is valid.
5. The verified heuristic is more informative than  $h_0$  because it expanded fewer nodes while having the same solution costs. The evidence supports using the proposed heuristic.

| Word pair    | Cost measure | From question 2 | Solution costs | Numbers of expanded nodes |
|--------------|--------------|-----------------|----------------|---------------------------|
| WHY -> ASK   | Frequency    | Yes             | 38.86          | 755                       |
| WHY -> ASK   | Frequency    |                 | 38.86          | 634                       |
| BEAR -> DUCK | Frequency    | Yes             | 38.24          | 720                       |
| BEAR -> DUCK | Frequency    |                 | 38.24          | 257                       |

**AI use:**

I had Claude help me write function `h`, debug and fix my logic if I coded it wrong. I also had it explain some concepts and explain things I was confused about. I also used it to check my work to see if I did anything wrong and had it explain why I was wrong.

**External resources:**

- <https://www.geeksforgeeks.org/python/self-in-python-class/>